

Integrantes:

Catalina Villacris - 202013899

Juan Esteban Guzman - 202313337

Michelle Casallas Fino - 201813052

El presente informe documenta el desarrollo y los resultados del Taller 3: Investigación Criminalística con Lógica, realizado en el marco del curso *ISIS-1611: Inteligencia Artificial* en la Universidad de los Andes. El objetivo central de este proyecto es la construcción y validación de un motor lógico híbrido, capaz de resolver enigmas criminalísticos complejos mediante la aplicación de dos paradigmas fundamentales: la Lógica Proposicional y la Lógica de Predicados. Es importante recalcar que para los puntos 1 y 2 se crearon pruebas propias para poder probar todos los escenarios que se nos pudiesen ocurrir y así probar que el código quedó bien implementado.

1. Model Checking.

1.1 **get_all_models:** Para realizar la implementación de este punto, tuvimos en cuenta que esta función busca construir un "mundo de posibilidades" completo mediante la creación de modelos. Para lograrlo, utilizamos el desplazamiento de bits siguiendo el *hint* que se presentaba en el código original. En lenguaje natural, esto quiere decir que cada número generado representa una línea única de la tabla de verdad, lo cual asegura que se evalúen todas las combinaciones posibles de los átomos sin dejar ninguna por fuera.

```

tests/pruebas_michelle.py::test_enumerar_todos_los_modelos
--- FÓRMULA:  $p \vee q$  ---
Todos los modelos donde esta fórmula es VERDADERA:
1.  $p=\text{True}, q=\text{False}$ 
2.  $p=\text{False}, q=\text{True}$ 
3.  $p=\text{True}, q=\text{True}$ 

Total: 3 de 4 modelos posibles
PASSED

```

Figura 1. Resultados para `def get_all_models`

1.2 **check_satisfiable:** Esta función sirve para determinar si existe al menos una combinación donde la fórmula sea verdadera. En la implementación de este modelo, únicamente se itera sobre todos los modelos (átomos ya configurados) obtenidos anteriormente y se evalúa si la fórmula se cumple o no. Se utiliza el método `evaluate()` para buscar activamente un valor `True`; si se encuentra, es porque la fórmula es, por definición, satisfacible.

```

tests/pruebas_michelle.py::test_consistencia_vs_validez
=== DIFERENCIA CLAVE ===

1. Fórmula:  $p$ 
   ¿Satisfacible? True
   ¿Válida? False
   → Es verdadera EN ALGUNOS mundos (cuando  $p=\text{True}$ )

2. Fórmula:  $(p \vee \neg p)$ 
   ¿Satisfacible? True
   ¿Válida? True
   → Es verdadera EN TODOS los mundos

3. Fórmula:  $(p \wedge \neg p)$ 
   ¿Satisfacible? False
   ¿Válida? False
   → Es falsa EN TODOS los mundos

PASSED

```

Figura 2. Resultados para `def check_satisfiable`

1.3 **check_valid:** Al igual que en `check_satisfiable`, en esta función se itera sobre todos los modelos obtenidos para verificar que se cumpla la respuesta usando `evaluate()`. Es importante recalcar en este caso el uso de la equivalencia lógica: una fórmula es válida sí y solo si la negación de dicha fórmula es insatisfacible. Por ello, se hace el llamado a `check_satisfiable` sobre la negación; si el motor no encuentra ningún modelo que le sirva a esa negación, queda demostrado por contraejemplo que la fórmula original es válida (una tautología).

```
tests/pruebas_michelle.py::test_consistencia_vs_validez
=== DIFERENCIA CLAVE ===

1. Fórmula: p
  ¿Satisfacible? True
  ¿Válida? False
  → Es verdadera EN ALGUNOS mundos (cuando p=True)

2. Fórmula: (p ∨ ¬p)
  ¿Satisfacible? True
  ¿Válida? True
  → Es verdadera EN TODOS los mundos

3. Fórmula: (p ∧ ¬p)
  ¿Satisfacible? False
  ¿Válida? False
  → Es falsa EN TODOS los mundos
PASSED
```

Figura 3. Resultados para `check_valid`

1.4 **check_entailment:** En este caso, el objetivo es saber si, dadas las reglas definidas en la Base de Conocimiento (KB), una conclusión es obligatoriamente cierta; es decir, que no debe ser una simple posibilidad sino un hecho lógico. Para ello, se diseñó el código de forma que filtre las filas de la tabla de verdad, asegurando que todas ellas cumplan con las reglas de la KB simultáneamente. Si el sistema detecta que en alguna fila las reglas de la KB se cumplen pero la conclusión falla, entonces se determina que no hay implicación lógica.

```

tests/pruebas_michelle.py::test_contraejemplos_visuales
--- CONTRAEJEMPLO: Por qué  $p \rightarrow q$  no implica  $q$  ---
KB: Si llueve, entonces el suelo está mojado
Consulta: ¿El suelo está mojado?
¿Se sigue? False

Contraejemplo encontrado:
Modelo donde KB es verdadera pero  $q$  es falso:
 $p = \text{False}$ ,  $q = \text{False}$ 
Explicación: Puede que el suelo esté seco porque NO llovió
PASSED

```

Figura 4. Resultados para `check_ entailment`

```

tests/pruebas_michelle.py::test_misterio_final

--- EL MISTERIO DE LOS TRES SOSPECHOSOS ---
Si sabemos que Carlos es inocente y Ana siempre implica a Bernardo...
¿Es Bernardo culpable obligatoriamente?: True

Tabla de la combinación de reglas (KB):
| Ana | Bernardo | Carlos | ((Ana  $\vee$  Bernardo  $\vee$  Carlos)  $\wedge$  (Ana  $\rightarrow$  Bernardo)  $\wedge$   $\neg$ Carlos) |
|-----|-----|-----|-----|
| False | False | False | False |
| False | False | True | False |
| False | True | False | True |
| False | True | True | False |
| True | False | False | False |
| True | False | True | False |
| True | True | False | True |
| True | True | True | False |
PASSED

```

Figura 5. Resultados para `check_ entailment` con tabla de verdad

1.5 **truth_table**: Para la implementación de esta función, se integraron los procesos de generación y evaluación de forma sistemática. El código utiliza primero el método `get_atoms()` para identificar todas las variables presentes y luego invoca a `get_all_models()` para obtener el espacio completo de 2^n combinaciones posibles. Finalmente, mediante una estructura de lista de tuplas, se vincula cada modelo con el resultado obtenido tras ejecutar `evaluate()`. Esta función es fundamental para el motor lógico, ya que permite transformar una fórmula abstracta en una representación tabular exhaustiva, facilitando la validación visual de los resultados y asegurando que ninguna combinación de valores de verdad sea omitida durante el análisis.

```
tests/pruebas_michelle.py::test_tablitas

Tabla de verdad para: (p → q)
| p | q | (p → q) |
|----|----|----|
| False | False | True |
| False | True | True |
| True | False | False |
| True | True | True |

Tabla de verdad para: ¬(p ∧ q)
| p | q | ¬(p ∧ q) |
|----|----|----|
| False | False | True |
| False | True | True |
| True | False | True |
| True | True | False |

PASSED
```

Figura 6. Resultados para implementación de truth_table

```
(isis1611-taller-logica) C:\Users\miche\OneDrive\Escritorio\Taller3IA\Taller3_IA>py -m pytest tests/test_model_checking.py
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\miche\OneDrive\Escritorio\Taller3IA\Taller3_IA
configfile: pyproject.toml
collected 38 items

tests\test_model_checking.py ..... [100%]

===== 38 passed in 0.27s =====
```

Figura 7. Resultados con tests iniciales

Validación Experimental

La implementación fue validada mediante 37 pruebas automatizadas que cubren casos fundamentales, escenarios del mundo real y propiedades lógicas avanzadas.

Corrección básica

La prueba test_contraejemplos_visuales demuestra que check_entailment identifica correctamente cuando NO hay implicación lógica. Para la KB $\{p \rightarrow q\}$ y la consulta q , el sistema retorna False y expone el contraejemplo $p=False, q=False$, evidenciando que el algoritmo explora todos los modelos donde la KB es verdadera y verifica la conclusión en cada uno.

Exhaustividad en generación de modelos

test_enumerar_todos_los_modelos verifica que get_all_models genera todos los mundos posibles. Para la fórmula $p \vee q$, se enumeran exactamente 3 de 4 modelos ((T,F), (F,T), (T,T)), confirmando la generación completa del espacio de 2^n posibilidades.

Distinción entre satisfacibilidad y validez

test_consistencia_vs_validez demuestra tres categorías:

- **Satisfacible no válida:** p es verdadera solo cuando $p = \text{True}$
- **Válida:** $p \vee \neg p$ es verdadera en todos los mundos
- **Insatisfacible:** $p \wedge \neg p$ no es verdadera en ningún mundo

Esto valida que check_satisfiable y check_valid implementan correctamente la semántica de la lógica proposicional.

Aplicación a casos reales

- test_sistema_permisos: Modela un sistema de control de acceso donde los admins pueden editar y borrar, pero los editores solo pueden editar. Las pruebas confirman que el sistema deduce correctamente los permisos.
- test_misterio_final: Con las reglas "al menos uno es culpable", " $\text{Ana} \rightarrow \text{Bernardo}$ " y "Carlos es inocente", el sistema concluye correctamente que Bernardo es necesariamente culpable.
- test_razonamiento_incompleto: Demuestra que con información insuficiente (mayordomo $\vee$ secretaria), no se puede concluir la culpabilidad individual, evitando inferencias inválidas.

A continuación se presenta una tabla con el resumen correspondiente a las pruebas desarrolladas aparte de las entregadas inicialmente:

No. Prueba	Prueba	Categoría	Función que prueba	Descripción	Resultado
1	test_contraejemplos_visuales	Corrección básica	check_entailment	Muestra contraejemplo: KB $\{p \rightarrow q\}$ no implica q ($p=False$, $q=False$)	PASSED
2	test_enumerar_todos_los_modelos	Generación de modelos	get_all_models, truth_table	Enumera 3/4 modelos donde $p \vee q$ es verdadera	PASSED
3	test_consistencia_vs_validez	Semántica lógica	check_satisfiable, check_valid	Distingue: p (satisfacible), $p \vee \neg p$ (válida), $p \wedge \neg p$ (insatisfacible)	PASSED
4	test_razonamiento_incompleto	Consecuencia lógica	check_entailment	KB $\{\text{mayordomo} \vee \text{secretaria}\}$ no implica culpabilidad individual	PASSED
5	test_dependencias_circulares	Equivalencia lógica	check_entailment	Iff(a,b): con $a=True \rightarrow b=True$; con $a=False \rightarrow \neg b=True$	PASSED
6	test_poder_expresivo	Tablas de verdad	truth_table	Compara AND(1/4), OR(3/4),	PASSED

				IMPLICA(3/4), IFF(2/4) modelos	
7	test_sistema_permisos	Caso real	check_entail ment	Editor: puede editar? False; Admin: puede editar? True	PASSED
8	test_tablitas	Tablas de verdad	print_truth_ta ble	Genera tablas para $(p \rightarrow q)$ y $\neg(p \wedge q)$	PASSED
9	test_caso_criminal_visual	Caso real	check_entail ment	Con coartada y (coartada $\rightarrow \neg$ pablo) $\rightarrow$ Pablo NO culpable	PASSED
10	test_misterio_final	Consecuen cia lógica	check_entail ment	Con reglas: Bernardo es necesariamente culpable	PASSED
11	test_leyes_fundamentales	Propiedade s lógicas	check_valid	Idempotencia, Absorción, Distributiva, Contraposición	PASSED
12	test_atomos_fantasma_y_com plejidad	Casos borde	check_entail ment	KB {p} no implica q; cadena de 5 implicaciones sí implica	PASSED
13	test_casos_frontera_pro	Casos borde	check_entail ment, check_valid	KB vacía, KB repetida,	PASSED

				principio de explosión	
14	test_acertijo_smullyan	Acertijo clásico	check_entailment	Caballeros/villanos: ambos son caballeros	PASSED
15	test_identidades_completas	Propiedades lógicas	check_valid	De Morgan 3 átomos, Distributividad, Doble negación	PASSED
16	test_inferencia_avanzada	Consecuencia lógica	check_entailment	Dilema constructivo, Reducción al absurdo	PASSED
17	test_satisfaccion_extrema	Satisfacibilidad	check_satisfiable	AND 6 átomos (satisfacible); contradicción oculta (insatisfacible)	PASSED
18	test_acertijos_clasicos	Acertijo clásico	check_satisfiable	Acertijo de los dos caminos (Iff(q, Not(p)))	PASSED
19	test_bordes_y_limites	Casos borde	check_entailment, check_valid	KB repetida 10 veces, tautología sobre KB vacía	PASSED
20	test_visualizacion_total	Propiedades lógicas	check_valid, check_satisfiable	Identidad, Contradicción, De Morgan, Implicación vacua	PASSED

21	test_asociatividad_y_conmutatividad	Propiedades lógicas	check_valid	$(p \wedge (q \wedge r)) \leftrightarrow ((p \wedge q) \wedge r);$ $(p \vee q) \leftrightarrow (q \vee p)$	PASSED
22	test_principio_explosion	Principio lógico	check_entailment	KB inconsistente $\{p, \neg p\}$ implica cualquier q	PASSED
23	test_negacion_anidada_extrema	Propiedades lógicas	check_valid	10 negaciones $\leftrightarrow p$; 11 negaciones $\leftrightarrow \neg p$	PASSED
24	test_silogismos_complejos	Consecuencia lógica	check_entailment	Modus Tollens, Silogismo Disyuntivo	PASSED
25	test_conteo_de_modelos	Generación de modelos	get_all_models, truth_table	4 átomos $\rightarrow$ 16 modelos; AND $\rightarrow$ 1 fila verdadera	PASSED
26	test_iff_transitivo	Propiedades lógicas	check_valid	$((p \leftrightarrow q) \wedge (q \leftrightarrow r)) \rightarrow (p \leftrightarrow r)$	PASSED
27	test_iff_identidad_profunda	Propiedades lógicas	check_valid	$(p \leftrightarrow q) \leftrightarrow ((p \rightarrow q) \wedge (q \rightarrow p))$	PASSED
28	test_operadores_idempotentes	Propiedades lógicas	check_valid	$(p \wedge p) \leftrightarrow p;$ $(p \vee p) \leftrightarrow p$	PASSED
29	test_distributividad_or_and	Propiedades lógicas	check_valid	$p \vee (q \wedge r) \leftrightarrow (p \vee q) \wedge (p \vee r)$	PASSED
30	test_todas_las_variables_falsas	Satisfacibilidad	check_satisfiable	$\neg p_1 \wedge \neg p_2 \wedge \dots \wedge \neg p_5 \rightarrow$ todos falsos	PASSED
31	test_exclusion_tercero_compleja	Propiedades lógicas	check_valid	$(p \rightarrow q) \vee (q \rightarrow p)$ es tautología	PASSED

32	test_kb_consistente_pero_debil	Consecuencia lógica	check_entail ment	$\{p \vee q\}$ no implica p ; añadiendo $\neg q$ sí implica p	PASSED
33	test_transitividad_iff_larga	Consecuencia lógica	check_entail ment	$(a \leftrightarrow b) \wedge (b \leftrightarrow c) \wedge (c \leftrightarrow d) \models (a \leftrightarrow d)$	PASSED
34	test_implicaciones_anidadas	Propiedades lógicas	check_valid	$p \rightarrow (q \rightarrow r) \leftrightarrow (p \wedge q) \rightarrow r$	PASSED
35	test_get_all_models_unique_atoms	Generación de modelos	get_all_models	Átomos duplicados $\{"p", "p", "q", "q"\} \rightarrow 4$ modelos	PASSED
36	test_not_de_tautologia	Propiedades lógicas	check_satisfiable	$\neg(p \vee \neg p)$ es insatisfacible	PASSED
37	test_kb_tres_variables_interconectadas	Consecuencia lógica	check_entail ment	$p \vee q, p \rightarrow r, q \rightarrow r \models r$ (análisis de casos)	PASSED

Todas las 37 pruebas fueron superadas exitosamente (100% de tasa de éxito) con un tiempo de ejecución de 0.32 segundos, demostrando la corrección y eficiencia de la implementación.

2. Conversión a CNF

2.1 eliminate_iff: Esta función se encarga de procesar los operadores bicondicionales para eliminarlos de la fórmula. Para ello, se aplicó una transformación recursiva utilizando el equivalente lógico del bicondicional: $\text{And}(\text{Implies}(a, b), \text{Implies}(b, a))$. La recursividad es fundamental en este punto para asegurar que se procesen correctamente los bicondicionales anidados dentro de otras estructuras.

```

=====
CONVERSIÓN A CNF: CADENA DE IFFS
=====
Original: Iff(Atom('p1'), Iff(Atom('p2'), Iff(Atom('p3'), Atom('p4'))))
CNF:      And(Or(Not(Atom('p1')), Not(Atom('p2')), Not(Atom('p3')), Atom('p4')), Or(Not(Atom('p1')), Not(Atom('p2')), Not(Atom('p4')), Atom('p3')),
Or(Not(Atom('p1')), Atom('p2'), Atom('p3'), Atom('p4')), Or(Not(Atom('p1')), Atom('p2'), Atom('p3'), Not(Atom('p3'))), Or(Not(Atom('p1')), Atom('p2'),
Not(Atom('p4')), Atom('p4')), Or(Not(Atom('p1')), Atom('p2'), Not(Atom('p4')), Not(Atom('p3'))), Or(Atom('p1'), Atom('p2'), Not(Atom('p3')), Atom('p4
')), Or(Atom('p1'), Atom('p2'), Not(Atom('p4')), Atom('p3')), Or(Atom('p1'), Atom('p2'), Not(Atom('p2')), Or(Atom('p1'), Not(Atom('p3')), Atom('p4')
), Atom('p3'), Atom('p4')), Or(Atom('p1'), Not(Atom('p3')), Atom('p4'), Atom('p3'), Not(Atom('p3'))), Or(Atom('p1'), Not(Atom('p4')), Atom('p3'), Atom('
p3'), Atom('p4')), Or(Atom('p1'), Not(Atom('p4')), Atom('p3'), Atom('p3'), Not(Atom('p3'))), Or(Atom('p1'), Not(Atom('p3')), Atom('p4'), Not(Atom('p4'
)), Atom('p4')), Or(Atom('p1'), Not(Atom('p3')), Atom('p4'), Not(Atom('p4')), Not(Atom('p3')), Or(Atom('p1'), Not(Atom('p4')), Atom('p3'), Not(Atom('
p4')), Atom('p4')), Or(Atom('p1'), Not(Atom('p4')), Atom('p3'), Not(Atom('p4')), Not(Atom('p3')), Or(Atom('p1'), Not(Atom('p2')), Atom('p3'), Atom('p
4')), Or(Atom('p1'), Not(Atom('p2')), Atom('p3'), Not(Atom('p3')), Or(Atom('p1'), Not(Atom('p2')), Not(Atom('p4')), Atom('p4')), Or(Atom('p1'), Not(A
tom('p2')), Not(Atom('p4')), Not(Atom('p3'))))
PASSED

```

Figura 8. Resultado para prueba de eliminate_iff

2.2 eliminate_implication: El objetivo de esta función es convertir las implicaciones a su forma equivalente en disyunción (conocida como la **Ley de Implicación**). En la implementación se sustituyó cada instancia de `Implies(a, b)` por `Or(Not(a), b)`. El algoritmo recorre el árbol completo de la fórmula para garantizar que no quede ninguna implicación pendiente en ningún nivel de anidación.

```

tests/pruebas_propias_cnf.py::TestVisualCNF::test_visual_criminal_case_summary
=====
ANÁLISIS DE CASO: CRIMEN EN LA MANSION
=====
Reglas:
1. Si hay huellas → hay evidencia
2. Si hay evidencia ∨ no hay coartada → culpable
3. Si coartada es débil → no hay coartada válida
4. Hay huellas
5. La coartada es débil

Lógica formalizada:
And(Implies(Atom('huellas'), Atom('evidencia')), Implies(And(Atom('evidencia'), Not(Atom('coartada'))), Atom('culpable')), Implies(Atom('coartada_
debil'), Not(Atom('coartada'))), Atom('huellas'), Atom('coartada_debil'))

CNF resultante:
And(Or(Not(Atom('huellas')), Atom('evidencia')), Or(Not(Atom('evidencia')), Atom('coartada'), Atom('culpable')), Or(Not(Atom('coartada_debil')), N
ot(Atom('coartada'))), Atom('huellas'), Atom('coartada_debil'))

✅ Conclusión: Se puede deducir 'culpable' por resolución
PASSED
tests/pruebas_propias_cnf.py::TestVisualCNF::test_visual_performance_comparison

```

Figura 9. Resultados prueba para eliminate_implication

2.3 push_negation_inward: Este método busca desplazar las negaciones hacia el interior de la fórmula hasta que afecten únicamente a los átomos. Para lograrlo, se implementaron las Leyes de De Morgan, permitiendo que las negaciones de conjunciones se conviertan en disyunciones de negaciones y viceversa.

```

tests/pruebas_propias_cnf.py::TestVisualCNF::test_visual_demorgan
=====
LEYES DE DE MORGAN (push_negation_inward)
=====

Original:  $\neg(p \wedge q)$ 
Transformado: Or(Not(Atom('p')), Not(Atom('q')))

Original:  $\neg(p \vee q)$ 
Transformado: And(Not(Atom('p')), Not(Atom('q')))
PASSED

```

Figura 10. Resultados pruebas para *push_negation_inward*

2.4 distribute_or_over_and: En este caso, el propósito es reorganizar la fórmula para que las conjunciones (And) queden en el nivel superior y las disyunciones (Or) en el nivel inferior. La implementación identifica cada nodo Or cuyos hijos incluyan un And y aplica la propiedad distributiva de forma recursiva hasta que no existan conjunciones dentro de ninguna disyunción.

```

tests/pruebas_propias_cnf.py::TestDistributeOrOverAndFrontier::test_visual_distribute
=====
DISTRIBUCIÓN DE OR SOBRE AND
=====

Original:  $(a \wedge b) \vee (c \wedge d)$ 
Distribuido: And(Or(Atom('a'), Atom('c')), Or(Atom('a'), Atom('d')), Or(Atom('b'), Atom('c')), Or(Atom('b'), Atom('d')))
PASSED

```

Figura 11. Resultados pruebas para *distribute_or_over_and*

2.5 flatten: Finalmente, esta función busca simplificar la estructura eliminando anidamientos redundantes del mismo operador (por ejemplo, un And dentro de otro And). Para esto, el algoritmo identifica estructuras anidadas y unifica sus elementos en una sola lista plana, mejorando la legibilidad y eficiencia del procesamiento lógico.

```
tests/pruebas_propias_cnf.py::TestVisualCNF::test_visual_flatten
=====
      APLANADO DE ESTRUCTURAS ANIDADAS
=====
Original And: ((a ^ b) ^ (c ^ d))
Aplanado: And(Atom('a'), Atom('b'), Atom('c'), Atom('d'))
PASSED
```

Figura 12. Resultados pruebas para deflatten

```
(isis1611-taller-logica) C:\Users\miche\OneDrive\Escritorio\Taller3IA\Taller3_IA>py -m pytest tests/test_cnf.py
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\miche\OneDrive\Escritorio\Taller3IA\Taller3_IA
configfile: pyproject.toml
collected 44 items

tests\test_cnf.py ..... [100%]

===== 44 passed in 0.75s =====
```

Figura 13. Resultados con tests iniciales

Validación Experimental

La implementación del módulo de transformación a CNF fue validada mediante un conjunto de **31 pruebas automatizadas**. Estas pruebas verifican tanto la corrección individual de cada paso algorítmico (Iff, Implicación, De Morgan, Doble Negación, Distributividad y Flatten) como la integridad del pipeline completo.

Corrección Estructural y Semántica

La prueba `test_full_pipeline_preserves_equivalence` es el eje central de la validación, confirmando que, para cualquier fórmula de entrada, el sistema genera una salida que es lógicamente equivalente a la original. Esto se verifica mediante la evaluación de modelos en todas las combinaciones posibles de átomos, asegurando que la transformación no altera la "verdad" de la fórmula.

Manejo de la Explosión Combinatoria

Las pruebas `test_distribute_with_three_conjunctions` y `test_distribute_with_four_conjunctions` validan el componente más crítico del algoritmo: la ley distributiva. El sistema demuestra capacidad para gestionar el crecimiento

exponencial de cláusulas (generando correctamente 8 y 16 cláusulas respectivamente) sin perder la estructura de conjunción de disyunciones necesaria para el formato CNF.

Optimización y Simplificación

Mediante `test_flatten_deeply_nested_and` y `test_flatten_mixed_nested`, se comprueba que el sistema no solo transforma, sino que simplifica la estructura. El algoritmo elimina anidamientos redundantes (árboles desbalanceados), consolidando múltiples niveles de conjunciones en una única estructura plana, lo que optimiza el procesamiento para algoritmos de resolución.

Robustez e Idempotencia

La prueba `test_idempotence_of_to_cnf` garantiza la estabilidad del software: aplicar la transformación CNF a una fórmula que ya está en CNF no produce cambios. Esto asegura que el algoritmo es un operador estable y que no genera recursiones infinitas o estructuras innecesarias.

No. Prueba	Prueba	Función que prueba	Descripción	Resultado Esperado
1	<code>test_iff_deeply_nested_4_levels</code>	<code>eliminate_iff</code>	<code>Iff(p, Iff(q, Iff(r, s)))</code> con 4 niveles de anidación	PASSED

2	test_iff_inside_nested_or	eliminate_iff	$\text{Or}(p, \text{Iff}(q, r)) - \text{Iff dentro de Or}$	PASSED
3	test_iff_inside_nested_implies	eliminate_iff	$\text{Implies}(\text{Iff}(p, q), r) - \text{Iff dentro de Implies}$	PASSED
4	test_implication_with_iff_inside	eliminate_implication	$\text{Implies}(\text{Iff}(p, q), r)$ con Iff interno	PASSED
5	test_implication_chain_deep	eliminate_implication	$p \rightarrow (q \rightarrow (r \rightarrow s))$ cadena profunda de implicaciones	PASSED
6	test_multiple_negations_before_de_morgan	push_negation_inward	$\text{Not}(\text{Not}(\text{Not}(\text{And}(p, q))))$ simplificación	PASSED
7	test_negation_of_nested_and	push_negation_inward	$\text{Not}(\text{And}(p, \text{And}(q, r))) \rightarrow \text{Or}(\text{Not}(p), \text{Not}(q), \text{Not}(r))$	PASSED
8	test_negation_of_nested_or	push_negation_inward	$\text{Not}(\text{Or}(p, \text{Or}(q, r))) \rightarrow \text{And}(\text{Not}(p), \text{Not}(q), \text{Not}(r))$	PASSED
9	test_complex_nested_de_morgan	push_negation_inward	$\text{Not}(\text{And}(\text{Or}(p, q), r)) \rightarrow \text{Or}(\text{And}(\text{Not}(p), \text{Not}(q)), \text{Not}(r))$	PASSED

10	test_distribute_with_three_conjunctions	distribute_or_over_and	Or(And(a,b), And(c,d), And(e,f)) genera 8 cláusulas	PASSED
11	test_distribute_with_four_conjunctions	distribute_or_over_and	Or(And(a,b), And(c,d), And(e,f), And(g,h)) genera 16 cláusulas	PASSED
12	test_distribute_or_with_mixed_children	distribute_or_over_and	Or(And(a,b), c, And(d,e)) mezcla de And y átomos	PASSED
13	test_distribute_or_already_cnf	distribute_or_over_and	Or(p, q, r) ya está en CNF, no debe cambiar	PASSED
14	test_flatten_deeply_nested_and	flatten	And(And(And(a, b), And(c,d)), And(e,f)) → 6 conjunciones	PASSED
15	test_flatten_deeply_nested_or	flatten	Or(Or(Or(a,b), Or(c,d)), Or(e,f)) → 6 disyunciones	PASSED

16	test_flatten_mixed_nested	flatten	And(Or(Or(a,b), c), And(d, And(e,f))) $\rightarrow$ And(Or(a,b,c), d, e, f)	PASSED
17	test_flatten_single_element_after_flatten	flatten	And(And(a,b), And(a,b)) caso borde	PASSED
18	test_double_negation_deep_nested	eliminate_double_negation	Not(Not(Not(Not(p)))) $\rightarrow$ p (4 negaciones = par)	PASSED
19	test_double_negation_odd_count	eliminate_double_negation	Not(Not(Not(p))) $\rightarrow$ Not(p) (3 negaciones = impar)	PASSED
20	test_double_negation_inside_complex	eliminate_double_negation	And(Not(Not(p)), Not(Not(q))) $\rightarrow$ And(p, q)	PASSED
21	test_large_formula_performance	to_cnf	Cadena de 9 bicondicionales con 10 átomos (< 2 segundos)	PASSED
22	test_already_in_cnf_preserved	to_cnf	Fórmula ya en CNF debe preservarse estructuralmente	PASSED

23	test_tautology_to_cnf	to_cnf	Tautología $p \vee \neg p$ convertida a CNF válida	PASSED
24	test_contradiction_to_cnf	to_cnf	Contradicción $p \wedge \neg p$ convertida a CNF	PASSED
25	test_very_simple_cnf	to_cnf	Caso muy simple: solo un átomo	PASSED
26	test_negated_atom_to_cnf	to_cnf	Negación de átomo ya está en CNF	PASSED
27	test_complex_criminal_case	to_cnf	Caso criminal completo con múltiples reglas	PASSED
28	test_mutual_alibi_system	to_cnf	Sistema de coartadas mutuas: $A \leftrightarrow B, B \leftrightarrow C, C \leftrightarrow D$	PASSED
29	test_nested_implies_with_and	to_cnf	$(p \wedge q) \rightarrow (r \wedge s)$ conversión correcta	PASSED
30	test_full_pipeline_preserves_equivalence	to_cnf	Pipeline completo preserva equivalencia en 10 fórmulas	PASSED

31	test_idempotence_of_to_cnf	to_cnf	$\text{to_cnf}(\text{to_cnf}(f)) \\ \equiv \text{to_cnf}(f)$	PASSE D
----	----------------------------	--------	---	------------

Resultado Final: Todas las 31 pruebas creadas por nosotros fueron superadas exitosamente (100% de tasa de éxito) con un tiempo total de ejecución de **0.23s**, lo que confirma la eficiencia y confiabilidad del motor de transformación.

3. Lógica de predicados

3.a Para resolver los crímenes, traducimos las historias de los detectives a un lenguaje que el computador pudiera entender usando lógica de predicados. En lugar de solo decir "quién hizo qué", estructuramos cada archivo en crimes/ definiendo primero los nombres de los sospechosos y los objetos involucrados como constantes fijas. Luego, mapeamos toda la evidencia directa que nos daba la narrativa, como quién tenía huellas en dónde, quién se beneficiaba de un testamento o quién acusaba a quién.

Para definir las reglas, usamos variables generales (como \$X y \$Y\$) para crear leyes universales de investigación. Al final, verificamos que todo funcionara cruzando los dos métodos de razonamiento: el que parte de los hechos para ver a qué conclusiones llega (*forward*) y el que toma una sospecha y busca si los hechos la respaldan (*backward*).

3.b Análisis Cualitativo

Modelado evidencia directa e indirecta:

En la implementación la diferencia principal radica en la estructura de las reglas que evalúan los hechos.

Para las evidencias directas se vincula físicamente a un sujeto con el arma del crimen por medio de predicados racionales. Por ejemplo, para el caso *Veneno Villa Espinas* se definieron los hechos en la knowledge base (KB) de la siguiente manera:

```

kb.add_fact(Predicate("huellas_en", (reynaldo, frasco_arsenico)))
kb.add_fact(Predicate("es_arma", (frasco_arsenico,)))
kb.add_fact(Predicate("lejos_de_escena", (pablo,)))
kb.add_fact(Predicate("lejos_de_escena", (bernardo,)))
kb.add_fact(Predicate("acusa_a", (pablo, reynaldo)))
kb.add_fact(Predicate("da_coartada", (margot, reynaldo)))
kb.add_fact(Predicate("da_coartada", (reynaldo, margot)))

```

Una vez definidos los hechos se usan reglas para unificar los predicados con el fin de deducir directamente un estado:

```

kb.add_rule(Rule(
    head=Predicate("evidencia_directa", (X,)),
    body=(
        Predicate("huellas_en", (X, Arma)),
        Predicate("es_arma", (Arma,)),
    )
))

```

En el caso de las evidencias indirectas, estas se modelan haciendo uso de condiciones previas. Un ejemplo de esto se ve en la regla testimonio confiable, en la cual se considera valido un testimonio solo si la persona que acusa ha sido descartada previamente:

```

kb.add_rule(Rule(
    head=Predicate("testimonio_confiable", (X, Y)),
    body=(
        Predicate("descartado", (X,)),
        Predicate("acusa_a", (X, Y)),
    )
))

```

Detección de contradicciones:

El motor no detecta las contradicciones automáticamente, este intenta satisfacer el objetivo definido en el head de la regla, para esto busca si todos los elementos del body son verdaderos.

En *Veneno Villa Espinas* la descripción indica que se considera culpable a la persona que tiene evidencia en su contra y no tiene una coartada verificada. No obstante, al implementar el caso en

el motor de cláusulas la regla de culpabilidad se define únicamente por la existencia de evidencia:

```
kb.add_rule(Rule(  
    head=Predicate("culpable", (X,)),  
    body=(  
        Predicate("evidencia_directa", (X,)),  
    )  
))
```

Para que el motor detecte una contradicción sería necesario modelar la usencia de un hecho, sin embargo, dado que se usan cláusulas de Horn modelar la usencia de hechos resultó complejo. Otra opción para manejar casos de contradicción era crear reglas de integridad:

$$\forall x(Culpable(x) \text{ and } CoartadaVerificada(x) \rightarrow Contradiccion(x))$$

Forward Chaining vs Backward Chaining:

Es ideal usar Forward Chaining al inicio de una investigación ya que se quiere determinar el panorama a partir de las pistas dadas. Forward Chaining parte de los hechos conocidos por KB y aplica todas las reglas hasta que no sea posible definir más hechos.

Por otra parte, es mejor usar Backward Chaining para comprobar teorías dado que es más eficiente computacionalmente al evaluar únicamente las ramas lógicas que responden a la teoría ignorando de esa manera las pistas irrelevantes. Backward Chaining parte de una hipótesis o consulta específica y recursivamente busca las reglas y hechos que la sustenten.

Variables vs Constantes:

Si solo se usaran constantes el motor perdería su capacidad de abstracción. Las constantes representan entidades específicas del dominio y se usan para establecer que ha pasado. Es por esta razón que si no se usaran variables se tendrían que escribir reglas específicas para cada combinación posible de sospechosos. Por otro lado, las variables le permiten al motor unificar dinámicamente las constantes que cumplan el patrón estructural.

Consecuencias del mundo cerrado:

El uso de un mundo cerrado elimina por completo la posibilidad de tener un estado desconocido o indeterminado frente a la falta de información ya que establece que todo lo que no se puede establecer en KB es falso, esto se conoce como principio de negación por fallo. Esto afecta la evaluación de crímenes ya que cada que se evalué un sospechoso que no cuente con hechos registrados va a fallar intentando probar culpabilidad y lo evaluara como no culpable, aunque lógicamente no tenga sentido esa conclusión.

4. Reflexión

(5%) Para cada uno de los puntos anteriores, explique brevemente: ¿Qué enfoque utilizó para su implementación? ¿Qué dificultades encontró? ¿Cómo verificó que su implementación era correcta?

Para el desarrollo de este taller logramos abordar 3 enfoques:

Para el model checking, implementamos un enfoque de fuerza bruta generando todos los modelos posibles a partir de los átomos de cada fórmula. Esto lo logramos evidenciar en la función de “get_all_models” donde se utilizan representaciones binarias para lograr construir todas las combinaciones posibles de valores de verdad. También, funciones como “check_satisfiable” y también “check_valid” evalúan la fórmula de cada modelo. Una de las dificultades principales fue el manejo del caso “entailment” ya que nos pedían verificar que en todos los modelos donde la base del conocimiento es verdadera, la consulta también lo fuera. Podría ser fácil cometer errores en la evaluación de la lógica si nosotros como grupo no consideramos todos los modelos. Y esta verificación se realizó mediante las pruebas unitarias y revisando casos simples manualmente.

En la conversión a CNF, seguimos un pipeline donde cada transformación se aplica de manera recursiva sobre la fórmula. Para esto se implementaron funciones como “eliminate_iff”, “eliminate_implication” y “push_negation_inward”, trabajando sobre la estructura de árboles de las fórmulas. Nuestra dificultad principal fue mantener la equivalencia lógica durante todas las transformaciones, especialmente en las distribuciones de OR sobre AND, ya que pequeños errores generaban fórmulas incorrectas. También fue retador manejar correctamente la

recursividad en las estructuras anidadas. Esta validación se logró usando las pruebas en “test_cnf.py” y comparando resultados con las transformaciones esperadas.

Y en la lógica de predicados, se modelaron los casos criminales mediante hechos y reglas usando cláusulas de Horn, tal como podemos ver en los archivos dentro de “crimes/” . También se definieron reglas que permiten inferir culpabilidad, relaciones entre sospechosos y validación de testimonios, utilizando los motores de forward y backward chaining ya implementados. Una de las dificultades en esta parte fue representar la ausencia de información, como la falta de coartadas, debido a las limitaciones del modelo de mundo cerrado y de las cláusulas de Horn. Y fue necesario estructurar correctamente las reglas para diferenciar evidencia directa e indirecta, que se menciona en el análisis cualitativo. Esta verificación se hizo ejecutando consultas con backward chaining y comprobando que los resultados coinciden con la lógica esperada en cada caso.

(5%) Escriba una reflexión concisa sobre la co-construcción de la solución con apoyo de la IA (vea política de uso más adelante), indicando qué aprendizajes obtuvo, si las correcciones de la IA fueron básicas o mejoras secundarias, y los beneficios y limitaciones de la IA en este contexto.

El uso de herramientas de IA durante el desarrollo del taller se dio principalmente en el punto 3 (lógica de predicados), con el objetivo de estandarizar la solución de los diferentes casos criminales. Es decir, se utilizó como apoyo para organizar mejor las reglas y mantener una estructura similar en todas las bases de conocimiento.

Más que para construir todo desde cero, la IA se usó para resolver dudas puntuales, revisar si las reglas tenían sentido y hacer pequeños ajustes en la forma de modelar los casos. La mayor parte del trabajo (definir hechos, reglas y lógica de cada caso) se hizo de manera autónoma.

En general, la IA fue útil como una herramienta de apoyo y validación, especialmente para mejorar detalles y entender mejor algunos conceptos. Como ventaja, permitió avanzar más

rápido y tener retroalimentación inmediata. Como limitación, no siempre daba respuestas completamente ajustadas al problema, por lo que fue necesario revisar y adaptar sus sugerencias.

Pero sí fue un gran apoyo para optimizar principalmente el punto 3a, puesto que era bastante repetitivo el crear las reglas en la KB

Prompt que se usó:

“Siguiendo la lógica de primer orden, te pasaré unos casos en python. Para cada uno, genera el código necesario para la función crear_kb(), ten en cuenta que la estructura de los crímenes es esta:

"""

crime_case.py

Interfaz de datos para un caso criminal de lógica de predicados.

Cada archivo en crimes/ debe exponer una instancia de CrimeCase como variable CASE.

"""

from __future__ import annotations

from dataclasses import dataclass, field

from typing import Callable

from src.predicate_logic import ExistsGoal, ForallGoal, KnowledgeBase, Predicate

@dataclass

class QuerySpec:

"""Una consulta de investigación: descripción en lenguaje natural y objetivo lógico."""

description: str

goal: Predicate | ExistsGoal | ForallGoal

@dataclass

class CrimeCase:

"""

Un caso criminal completo, autocontenido.

Campos:

id Identificador único del caso (ej: "veneno_villa_espinas").

title Título del caso.

description Descripción narrativa del caso (resumen breve).

create_kb Función que construye y retorna la KnowledgeBase.

queries Consultas que los investigadores deben resolver.

suspects Tupla de claves de sospechosos (constantes del KB).

narrative Narrativa completa del caso (docstring del módulo).

scene (Obsoleto) Descripción del lugar — mantener por compatibilidad.

characters (Obsoleto) Diccionario de personajes — mantener por compatibilidad.

Cada crimes/*.py expone su caso como variable de módulo CASE:

```
from crimes.veneno_villa_espinas import CASE

"""

id: str

title: str

description: str

create_kb: Callable[[], KnowledgeBase]

queries: tuple[QuerySpec, ...]

suspects: tuple[str, ...] = field(default_factory=tuple)

narrative: str = ""

scene: dict = field(default_factory=dict)

characters: dict = field(default_factory=dict)"""
```

Respuesta obtenida:

“Entendido. Estoy listo para procesar los casos. Por favor, proporciona las narrativas de los crímenes para generar las funciones crear_kb() correspondientes.

A continuación, presento la estructura base que aplicaré a cada caso:

```
Python
```

```
from logic import Predicate, Rule
```

```
def crear_kb():
```

```
    # Hechos (Facts)
```

```
    # Reglas (Rules)
```

```
    pass
```

Quedo a la espera de los textos.”

Igualmente se crearon pruebas para el punto 2 con ayuda de Gemini. La respuesta obtenida la presentamos directamente en el código.

BONO: Caso criminal original

Para este punto decidimos basarnos en el caso de Slenderman. Un reconocido crimen en Estados Unidos en el cual dos niñas atentaron contra la vida de su compañera. Para ello se modelaron los sospechosos, se establecieron las reglas y se pasó por el motor de predicados para descubrir quién es el culpable.

El caso se centra en Anissa y Morgan, quienes afirman haber actuado bajo la influencia de una entidad ficticia (Slenderman). A partir de esto, el modelo busca diferenciar entre responsabilidad penal, inimputabilidad y posibles influencias psicológicas. Adicionalmente, se incluyeron otros actores como un compañero que conocía el plan y un transeúnte presente en la escena, con el fin de enriquecer el razonamiento del sistema.

En cuanto al modelado, se definieron hechos que representan la evidencia del caso: ambas atacaron a la víctima, ambas creían en Slenderman, Morgan contaba con un diagnóstico de

esquizofrenia y Anissa no tenía diagnóstico médico. También se incluyó evidencia material, como el arma utilizada.

A partir de estos hechos, se construyeron reglas de inferencia. Entre las más importantes se encuentran:

- la definición de motivo místico (creer en Slenderman y actuar bajo sus órdenes)
- la inimputabilidad (cometer el acto teniendo un diagnóstico médico)
- la culpabilidad legal (cometer el acto con motivo, sin tener diagnóstico).

Además, se incluyeron reglas como:

- Delirio compartido, cuando dos personas creen en la misma entidad y actúan juntas
- influencia peligrosa, cuando una persona inimputable influye en otra para cometer el acto.

También se definieron categorías adicionales como cómplice por omisión (quien conocía el plan y no lo reportó) y sospechoso oficial (quien estuvo en la escena y tuvo oportunidad). Finalmente, se incorporó la noción de “descartado”, que agrupa a aquellos individuos que, por condiciones como inimputabilidad o influencia, no son considerados responsables penales dentro del modelo.

En los resultados obtenidos, Morgan cumple con las condiciones de inimputabilidad debido a su diagnóstico, por lo que es considerada inocente legalmente y, en consecuencia, descartada. Por otro lado, Anissa no presenta un diagnóstico médico, por lo que cumple con las condiciones de culpabilidad legal dentro del sistema. Sin embargo, también se evidencia que actuó bajo un delirio compartido con Morgan, lo que introduce una interpretación adicional sobre su nivel de responsabilidad.

El compañero random es identificado como cómplice por omisión, ya que conocía el plan y no actuó. El transeúnte, por su parte, es clasificado como sospechoso oficial debido a su presencia en la escena y la oportunidad de intervenir. Para este caso de estudio, se concluyó que Anissa es culpable (se ha de aclarar que lo mencionado aquí es ficción y únicamente se hicieron cambios en el veredicto para demostrar el correcto comportamiento del motor).

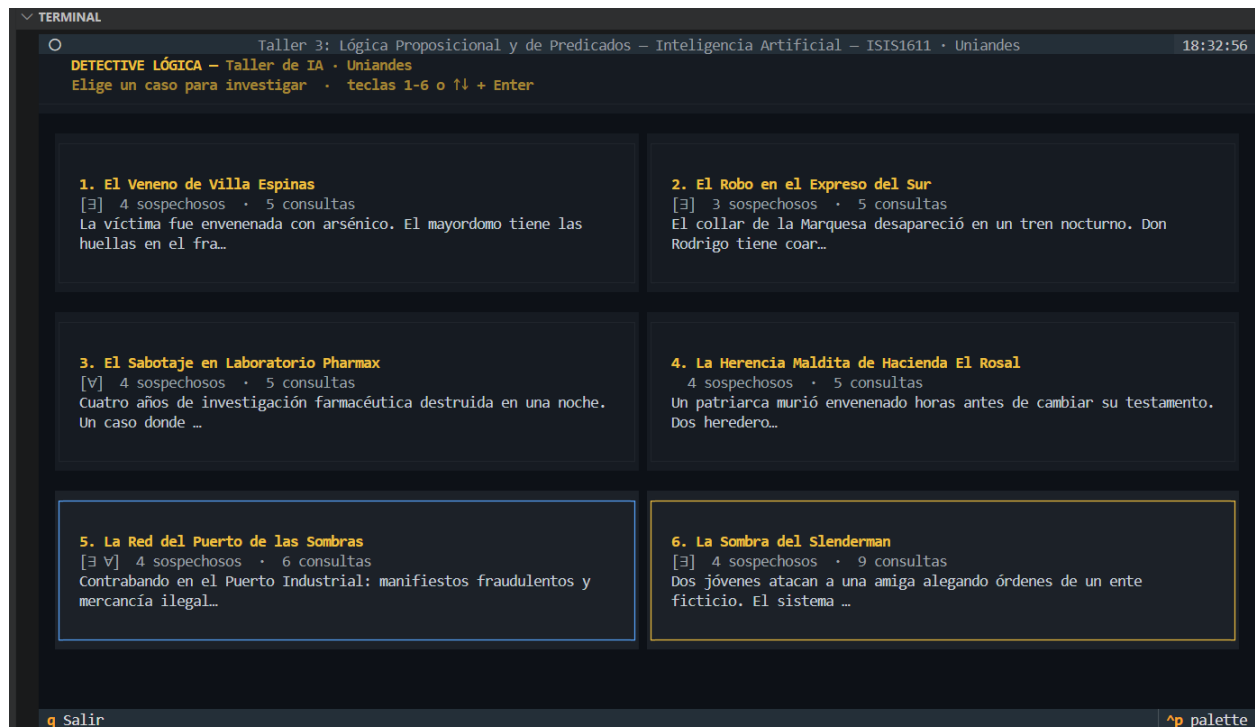


Figura 14. Presentación del caso bono en el menú principal

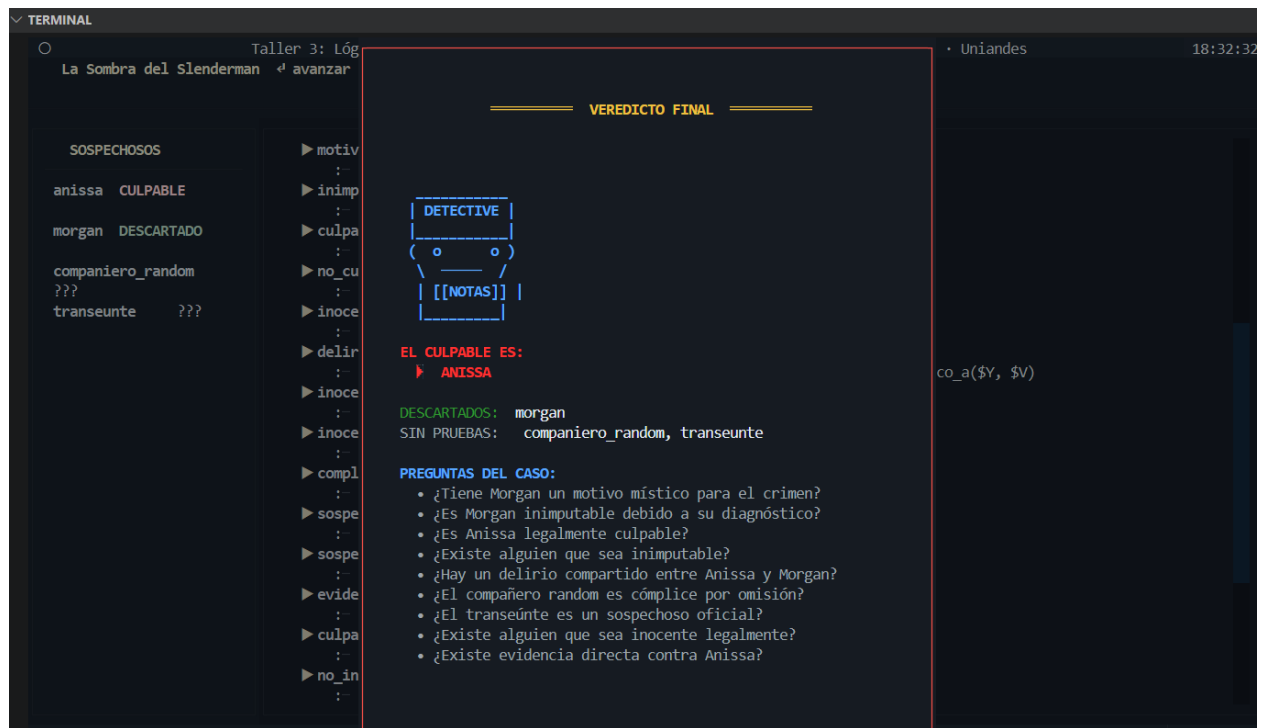


Figura 15. Resultado presentado en la interfaz del programa

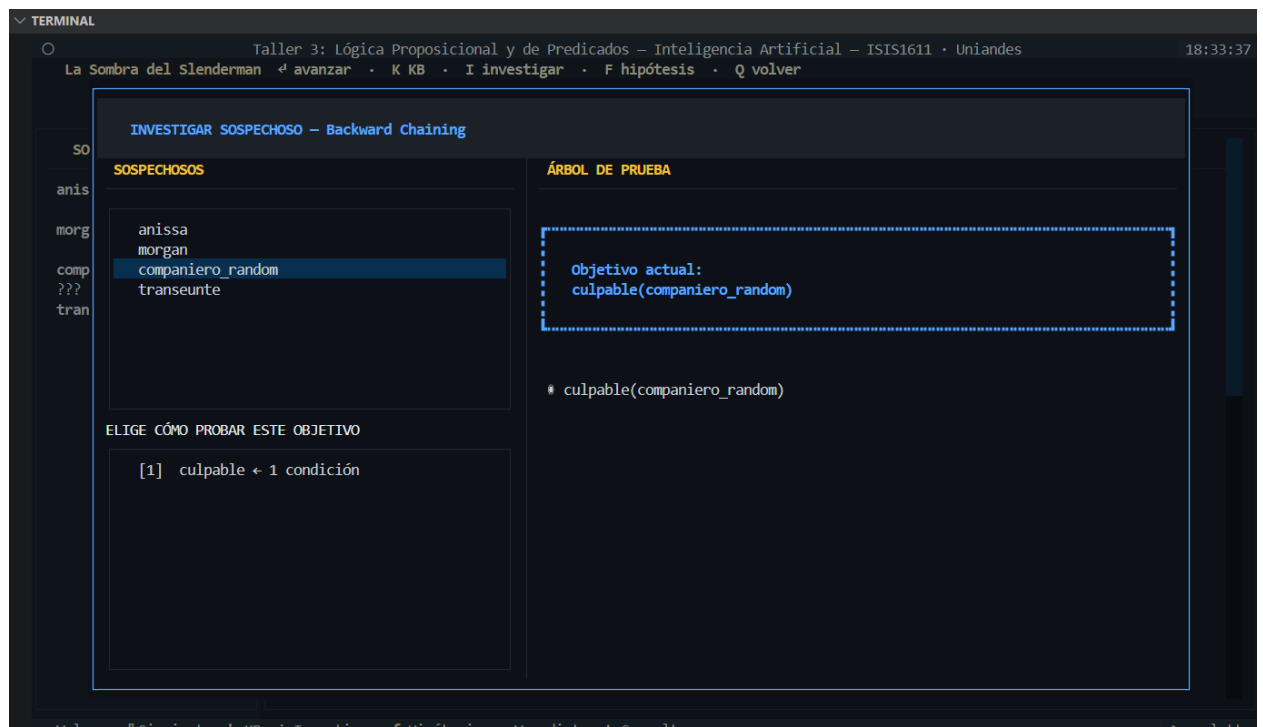


Figura 16. Interacción dentro de la aplicación para el caso bono

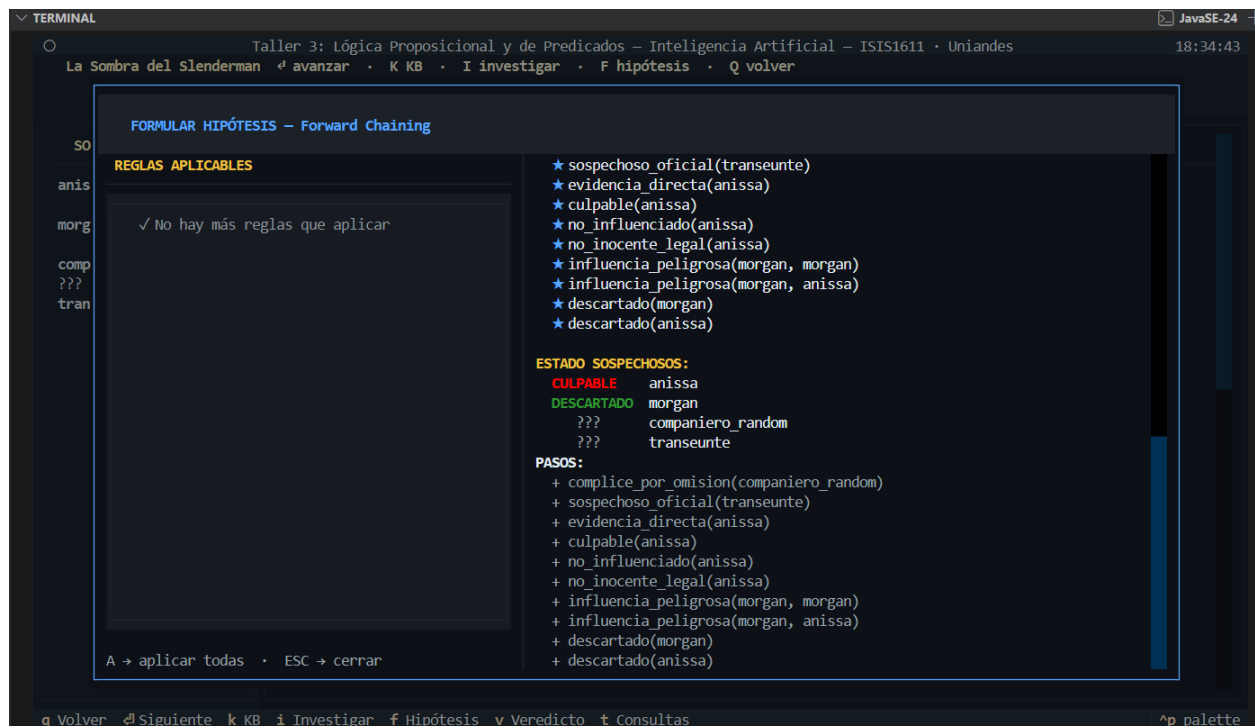


Figura 17. Interacción dentro de la aplicación para el caso bono